Functional Testing Report - Reschedule Appointment

User story: "As a patient, I want to reschedule an existing appointment to another available time slot, without having to cancel and book from scratch"

Tested with the test patient Silver Pelican

Summary

Recommendation: NO-GO

The new date and time are saved, but the rules around it are not enforced, and the patient sees wrong information after rescheduling

Main reasons:
- Appointments can be moved into a slot that is already taken (BUG-01)
- Appointments can be moved to the past, to year 0001 or to slots like "25:99" (BUG-02, BUG-05)
- Completed and cancelled appointments can be rescheduled (BUG-03)
- After rescheduling, the card shows the old time and a date one day earlier (BUG-04)

Also found, outside this story: cancelling an appointment does not save at all (BUG-09)

Tested manually in the UI, the API and the database: 18 test cases, 3 pass and 15 fail. Backed by a Postman collection and UI automation (Playwright + pytest). SQL, Postman, UI tests and AI notes are in the repository: https://github.com/Jaco25PL/medappoint_test.git

1. Test Strategy

What it should do: the patient picks an appointment, chooses a new date and a free slot, and confirms. The same appointment moves, the old slot is freed, the new one is taken, and nothing else changes (doctor, notes, payment)

There is no spec, so these are the rules I tested against:

- Only upcoming appointments (active or pending) can be rescheduled, never completed or cancelled ones
- The new date must be in the future and the slot must be free for that doctor
- No double booking, for the doctor or for the patient
- Inactive doctors can't get appointments
- Patients can only reschedule their own appointments
- The API must enforce all of this, even if the UI is bypassed

Scope: rescheduling from the UI and the API, what it affects (availability, list, dashboard, notifications, payments), and database checks of what was really saved. Out of scope: login and profile (bugs found there are reported as extras), performance and load, full security and accessibility audits, browsers other than Chrome

Risks, ordered by impact for the patient and by how likely they are after exploring (details in exploratory-findings.md):

Risk	Priority	Covered by
R1 Double booking	P1	TC-02, TC-03, Postman, SQL Q4
R2 Past, impossible or invalid date and slot	P1	TC-04 to TC-08, TC-14, Postman, UI tests
R3 Rescheduling completed or cancelled appointments	P1	TC-09, TC-10, Postman
R4 Date shown one day off	P1	TC-11, Postman, UI tests
R5 Rescheduling someone else's appointment	P1	TC-12, TC-13, Postman
R6 Old slot not freed, UI not updated	P2	TC-01, TC-15, Postman, UI tests
R7 Appointment with an inactive doctor	P2	TC-16, Postman, SQL Q3
R8 Side effects (payment, notification, updated_at)	P2	TC-17, Postman. Payments not covered
R9 Usability	P3	Exploration

How: exploratory testing first to find the risks, then test cases by priority (status, date boundaries, invalid input through the API). Each case is checked in three layers: the UI (what the patient sees), the API (whether the backend enforces the rule) and the database (what was really saved). Each case uses a fresh appointment created through the API and deleted afterwards. The database is shared with other candidates, so I only touched my own data

Exit criteria: all P1 and P2 cases run, every bug documented with evidence, no open P1 bugs to release

2. Test Cases

ID	Case	Result
TC-01	Happy path: reschedule to a free slot	FAIL (BUG-04, BUG-08, BUG-01)
TC-02	Slot already taken by the same doctor	FAIL (BUG-01)
TC-03	Patient already has an appointment at that time	FAIL (BUG-01)
TC-04	Past date	FAIL (BUG-02)
TC-05	Today, slot already passed	FAIL (BUG-02)
TC-06	Year 1	FAIL (BUG-02)
TC-07	Invalid slot ("25:99", spaces)	FAIL (BUG-05)
TC-08	Slot the doctor doesn't offer	FAIL (BUG-05)
TC-09	Completed appointment	FAIL (BUG-03)
TC-10	Cancelled appointment	FAIL (BUG-03)
TC-11	Date shown is the date picked	FAIL (BUG-04)
TC-12	Someone else's appointment	**PASS (403)**
TC-13	No token, bad token, appointment not found	**PASS (401, 401, 404)**
TC-14	Missing fields, wrong date format	FAIL (BUG-06). Missing fields are rejected fine
TC-15	List and dashboard update without reloading	FAIL (BUG-04)
TC-16	Inactive doctor	FAIL (BUG-07)
TC-17	Side effects	FAIL (BUG-08). Payments blocked: no paid appointment to test with
TC-18	Reschedule to the same date and time it already has	**PASS (no change)**

3. Bugs

Severity = how bad it is. Priority = how soon to fix it

Reschedule bugs

BUG-01 - You can reschedule into a slot that is already taken

- **Severity / Priority**: Critical / P1
- **Description**: nothing checks if the slot is free. Availability always returns the same 10 slots, even when they are booked
- **Steps**: book appointment A with a doctor at a date and time, then reschedule appointment B to that same date and time
- **Expected**: rejected, the slot is taken
- **Current**: 200, both appointments end up in the same slot. The patient can also be booked in two places at once (3 appointments at 09:00 on Nov 3 with 3 doctors)
- **Evidence**: Postman "Edge - Slot already taken". SQL Q4: 9 slots with more than one appointment, one of them with 10

BUG-02 - You can reschedule to the past, to year 0001, or to a slot that already passed today

- **Severity / Priority**: High / P1
- **Description**: the date input has no minimum date and the API doesn't check dates. For today, every slot is offered
- **Steps**: click Reschedule, pick a past date (or type 0001-01-01), pick a slot and confirm
- **Expected**: past dates can't be picked and the API rejects them
- **Current**: "Appointment rescheduled successfully" and the date is saved
- **Evidence**: UI tests (expected failures), Postman "Edge - Past date". SQL Q1: active appointments in 2002, 2000, 1999 and 0001

BUG-03 - Completed and cancelled appointments can be rescheduled

- **Severity / Priority:** High / P1
- **Description**: the Reschedule option shows for any status and the API accepts it, which messes up the medical and billing history
- **Steps**: click Reschedule on a completed appointment, pick a new date and slot and confirm
- **Expected**: no Reschedule option, and the API rejects it
- **Current**: saved. There are now completed appointments in the future
- **Evidence**: Postman "Appointment states". SQL Q1: two completed appointments with future dates

BUG-04 - After rescheduling, the card shows the old time and a date one day earlier

- **Severity / Priority**: High / P1
- Description: the reschedule response sends back the old data, and dates come as UTC midnight ("2027-07-09T00:00:00.000Z"), which in GMT-3 is the day before. The patient can go on the wrong day
- **Steps**: reschedule an appointment from Jul 9 11:30 to Jul 10 15:00 and look at the card
- **Expected**: the card shows 10/7/2027 15:00
- **Current**: the card shows 8/7/2027 11:30. After reloading the time is right but the date is still one day off. The dashboard doesn't update either
- **Evidence**: UI test "reschedule to free slot" (expected failure), Postman "Reschedule to a free slot"

BUG-05 - Invalid time slots are accepted

- **Severity / Priority**: Medium / P2
- **Description**: the API saves any text as a time slot, it only rejects an empty one
- **Steps**: reschedule an appointment with time_slot "25:99" (also tried " ", "12:00", "' OR '1'='1")
- **Expected**: 400, the slot must be one the doctor offers
- **Current**: 200, the value is saved as is
- **Evidence**: Postman "Edge - Time slot out of range (25:99)"

BUG-06 - Bad input crashes the API or changes the date without telling you

- **Severity / Priority**: Medium / P2
- **Description**: the API doesn't validate the input before it reaches the database
- **Steps**: reschedule with date "not-a-date", with a broken JSON body, with appointment id 2147483648, and with date February 30
- **Expected**: 400 with a clear message (404 for the id)
- **Current**: 500 for the first three. 2147483648 is one over the PostgreSQL integer limit (32-bit, max 2147483647). February 30 returns 200 and is saved as March 2
- **Evidence**: Postman "Edge - Date is not a date", the rest tried by hand

BUG-07 - Appointments with an inactive doctor can be booked and rescheduled

- **Severity / Priority:** Medium / P2
- **Description**: the API doesn't check if the doctor is active, and availability returns slots for him
- **Steps**: book an appointment with Dr. Pedro Sanchez (inactive), then reschedule it
- **Expected**: both are rejected
- **Current**: 201 and 200
- **Evidence**: Postman "inactive doctor" requests. SQL Q3: he still has 2 active future appointments

BUG-08 - No record of the change and no notification after a reschedule

- **Severity / Priority:** Low / P3
- **Description**: there is no way to know when an appointment was changed, and the patient is not told
- **Steps**: reschedule an appointment and read it again
- **Expected**: updated_at is the time of the reschedule, and a notification shows up
- **Current**: updated_at stays the same as created_at, and there is no notification
- **Evidence**: Postman "updated_at changed after the reschedule**"**

Outside the user story

BUG-09 - Cancelling does not save

- **Severity / Priority:** Critical / P1
- **Description**: the patient thinks they cancelled, but the appointment stays active and keeps the slot taken. It also blocks TC-10
- **Steps**: click Cancel on an appointment (or call PUT /api/appointments/{id}/cancel), then reload or read it again
- **Expected**: status is cancelled
- **Current**: the response says success and the card changes on screen, but it is still active in the API and the database
- **Evidence**: GET of the appointment after cancelling, and the database row

BUG-10 - Saving profile notes overwrites the name and you can't fix it

- **Severity / Priority:** High / P2
- **Description**: the patient's identity data gets corrupted and they can't restore it
- **Steps**: go to Profile, write a note and save, then go back and save again
- **Expected**: the note is saved and the name doesn't change
- **Current**: the first name becomes "Silver Pelican Hi, I am a doctor" and the last name is empty. Saving again returns 400, and sending the original name back returns 403
- **Evidence**: the profile screen. Another candidate's profile shows the same thing ("Jolly Koala Allergic to penicilin.")

BUG-11 - Notifications and header icons don't work

- **Severity / Priorit**y: Low / P3
- **Description**: the patient gets no reminders and some header icons are dead
- **Steps**: book an appointment for tomorrow and open Notifications, then click the bell and the gear in the header
- **Expected**: a reminder shows up, and both icons open something
- **Current**: no reminder, and the icons do nothing. The type shows as "appointment_reminder" instead of readable text
- **Evidence**: Notifications page

BUG-12 - Logging out does not invalidate the session token

- **Severity / Priority**: Medium / P2
- **Description**: logout only removes the token from the browser. The token keeps working on the server for 24 hours, so anyone who copied it, or uses the same computer, keeps access to the patient's appointments, personal data and payments
- **Steps**: log in, copy the token, call POST /api/auth/logout, then call GET /api/users/me with the same token
- **Expected**: 401 after logout
- **Current**: logout returns 200 "Logged out successfully", and the old token still returns 200 with the patient's data
- **Evidence**: API calls before and after logout

4. Improvements

- Pick the new slot from the doctor's real free slots instead of a free date input. That alone blocks most of BUG-01, 02 and 05 from the UI
- Show the doctor's name and specialty on each card
- Sort appointments with the next one first
- Refresh the list and dashboard right after a reschedule
- Use the app's own error messages. The browser ones show in the OS language ("Completa este campo" on a Spanish Windows)
- Link the "New Date" and "New Time" labels to their inputs, for screen readers
- Ask for confirmation before saving, and say so when nothing changed
- Add database rules as a safety net: no two active appointments in the same slot, real date and time types, a fixed list of statuses
- Keep the API docs in sync: they say dates are YYYY-MM-DD, the API returns full timestamps

5. Conclusion

The basics work: the new date and time are saved, and only the owner can change their appointment. Everything around it is missing. The API saves any date and slot, taken or not, past or not, real or not, and the UI doesn't stop it either. After a reschedule, the patient is even shown the wrong day. The data already in the database (double bookings, year 0001, completed appointments in the future) shows this is already happening. Most of it comes from one root cause: no business rules on the backend and no constraints in the database

With more time I would cover payments after a reschedule, concurrency (two patients taking the same slot at once) and the same negative cases for booking a new appointment

6. Go / No-Go

NO-GO

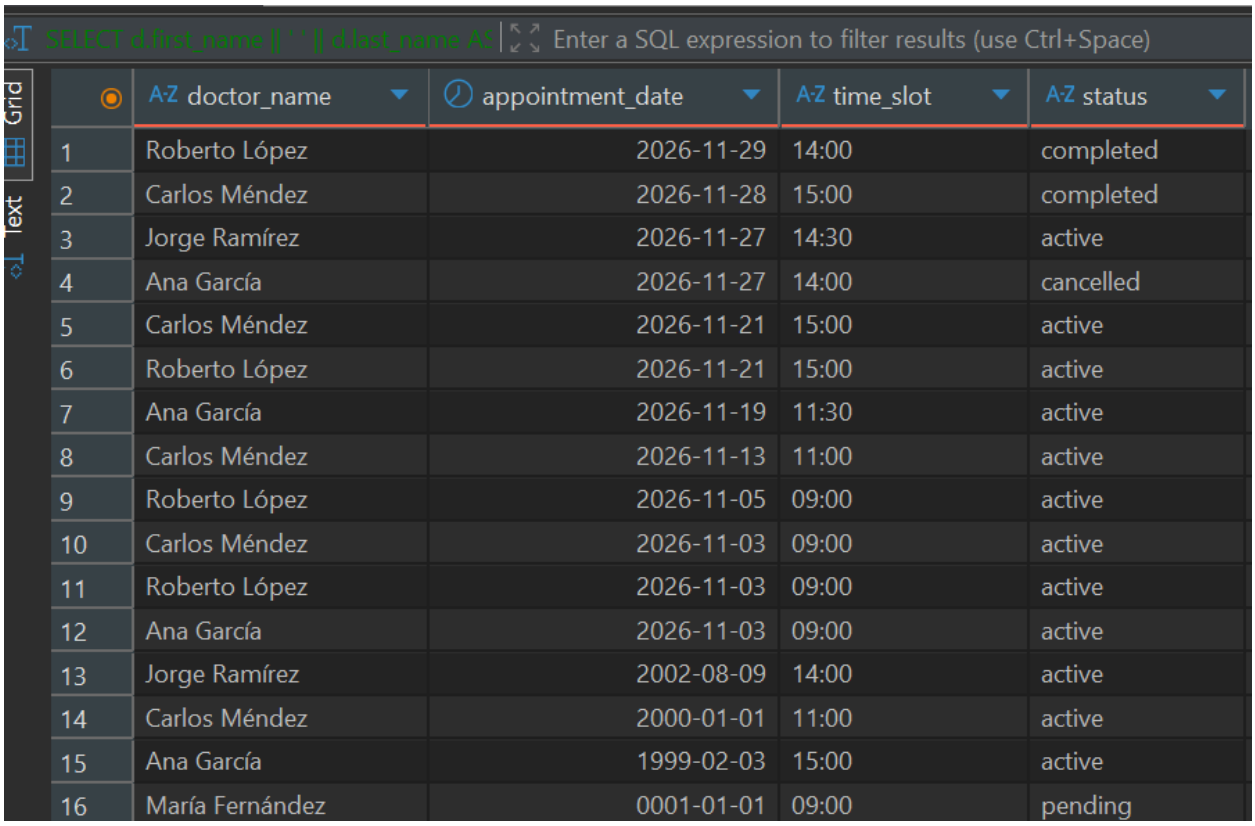
There are 4 open P1 bugs in this story (BUG-01 to BUG-04). Double booking and past dates break the core promise of the feature ("another available time slot"), and BUG-04 can send the patient on the wrong day

To change it to GO: fix BUG-01 to BUG-04 and run the tests again (the Postman collection and the UI tests cover them in a few minutes). BUG-05 to BUG-07 are the same kind of missing validation and should ideally go in the same fix. BUG-09 (cancel) is outside this story, but it should be fixed before any release of the appointments module

SQL queries, the Postman collection, UI tests, exploration notes and AI notes are in the repository (link in the summary)

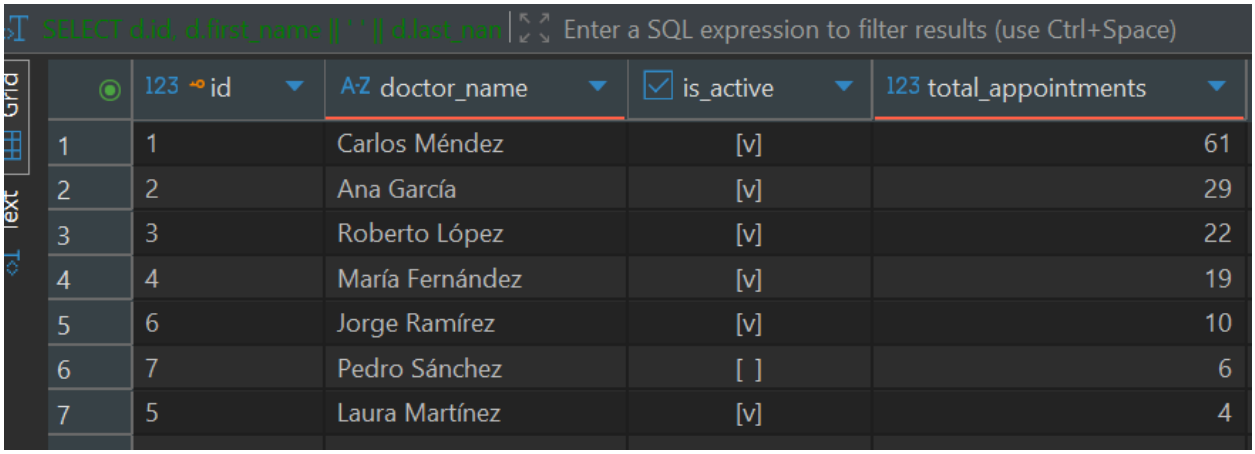
7. SQL

Q1 - My appointments
The test patient has 16 appointments. Some of them should not exist, completed appointments with future dates, appointments in the past (even year 0001), and three appointments at the same time with different doctors. They were created during the reschedule tests



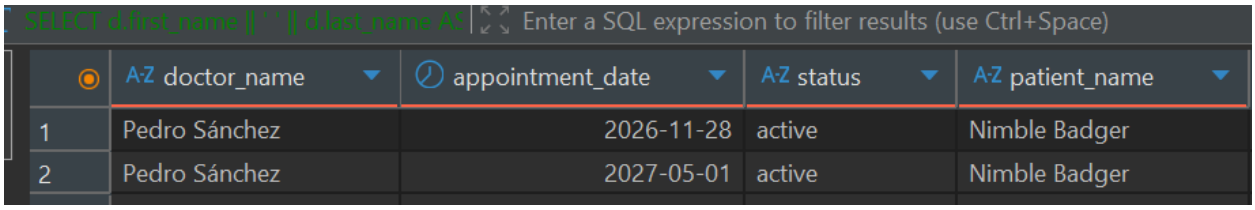
	doctor_name	appointment_date	time_slot	status
1	Roberto López	2026-11-29	14:00	completed
2	Carlos Méndez	2026-11-28	15:00	completed
3	Jorge Ramírez	2026-11-27	14:30	active
4	Ana García	2026-11-27	14:00	cancelled
5	Carlos Méndez	2026-11-21	15:00	active
6	Roberto López	2026-11-21	15:00	active
7	Ana García	2026-11-19	11:30	active
8	Carlos Méndez	2026-11-13	11:00	active
9	Roberto López	2026-11-05	09:00	active
10	Carlos Méndez	2026-11-03	09:00	active
11	Roberto López	2026-11-03	09:00	active
12	Ana García	2026-11-03	09:00	active
13	Jorge Ramírez	2002-08-09	14:00	active
14	Carlos Méndez	2000-01-01	11:00	active
15	Ana García	1999-02-03	15:00	active
16	María Fernández	0001-01-01	09:00	pending

Q2 - Appointments per doctor
All 7 doctors are listed. Dr. Pedro Sanchez is inactive but still has 6 appointments



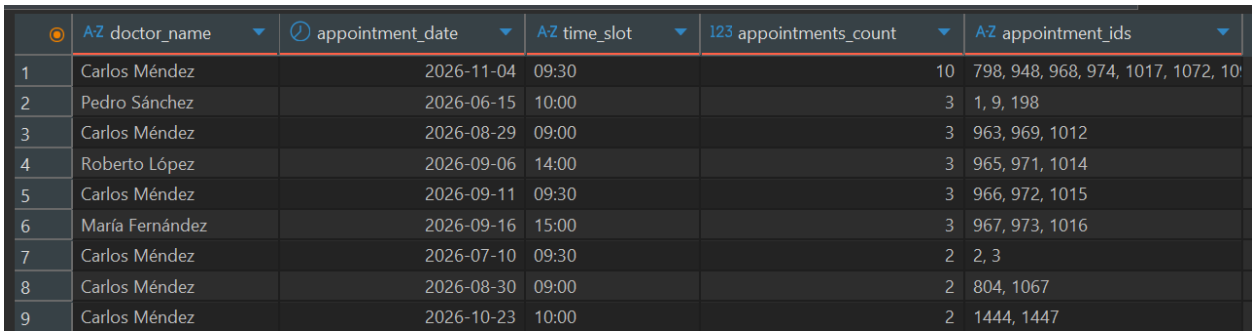
	id	doctor_name	is_active	total_appointments
1	1	Carlos Méndez	[v]	61
2	2	Ana García	[v]	29
3	3	Roberto López	[v]	22
4	4	María Fernández	[v]	19
5	6	Jorge Ramírez	[v]	10
6	7	Pedro Sánchez	[ ]	6
7	5	Laura Martínez	[v]	4

Q3. Future appointments with inactive doctors
Finding: Dr. Pedro Sanchez is inactive but still has 2 active future appointments. The patient expects an appointment that will not happen



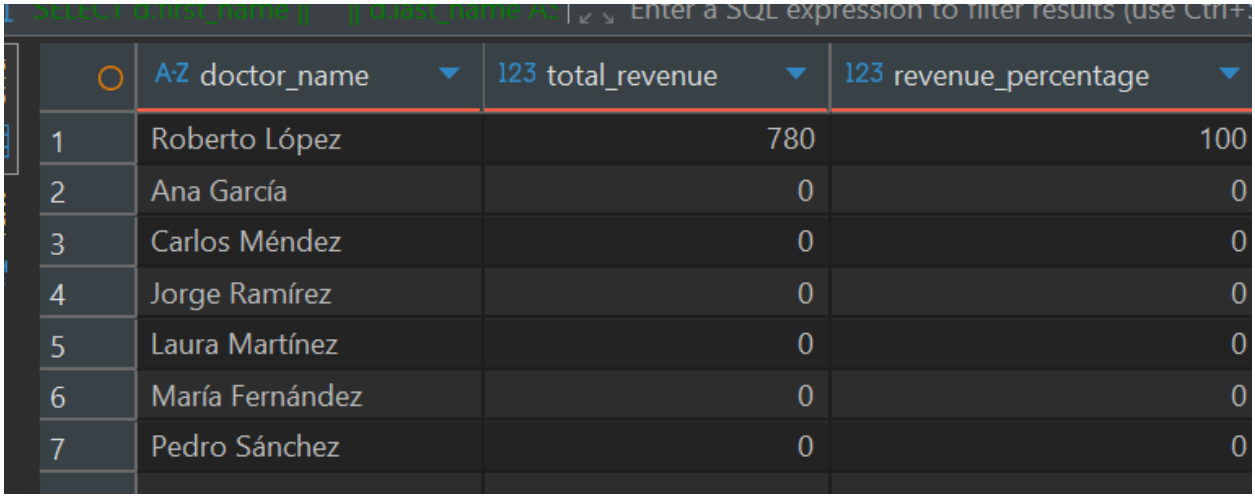
	doctor_name	appointment_date	status	patient_name
1	Pedro Sánchez	2026-11-28	active	Nimble Badger
2	Pedro Sánchez	2027-05-01	active	Nimble Badger

Q4 - Scheduling conflicts
Finding: 9 cases where a doctor has more than one appointment at the same date and time. The worst one has 10 appointments in a single slot. The system does not prevent double booking. Cancelled appointments are not counted.



	doctor_name	appointment_date	time_slot	appointments_count	appointment_ids
1	Carlos Méndez	2026-11-04	09:30	10	798, 948, 968, 974, 1017, 1072, 10
2	Pedro Sánchez	2026-06-15	10:00	3	1, 9, 198
3	Carlos Méndez	2026-08-29	09:00	3	963, 969, 1012
4	Roberto López	2026-09-06	14:00	3	965, 971, 1014
5	Carlos Méndez	2026-09-11	09:30	3	966, 972, 1015
6	María Fernández	2026-09-16	15:00	3	967, 973, 1016
7	Carlos Méndez	2026-07-10	09:30	2	2, 3
8	Carlos Méndez	2026-08-30	09:00	2	804, 1067
9	Carlos Méndez	2026-10-23	10:00	2	1444, 1447

Q5 - Revenue per doctor
The question says "completed" payments, but in the database they are stored as "paid". All the revenue (780) belongs to Dr. Roberto Lopez, the other doctors have no payments
Finding: the number of payments went from 14 to 13 during the same day. A payment disappeared, probably deleted together with its appointment through the API. Not confirmed



	doctor_name	total_revenue	revenue_percentage
1	Roberto López	780	100
2	Ana García	0	0
3	Carlos Méndez	0	0
4	Jorge Ramírez	0	0
5	Laura Martínez	0	0
6	María Fernández	0	0
7	Pedro Sánchez	0	0

8. Notes

What I would add in a real project:

- An AI skill that reads a screen and generates its page object file, with every locator already named following the project convention (btn_, txt_, ddl_...). That makes adding new screens fast and consistent. I didn't build it here because the reschedule flow lives on very few screens, so it would be overengineering for now.
- Bug reports drafted by AI: a command or workflow where I give a short description of the bug plus the evidence (screenshots or a failed test run), and it drafts the report in the team format: summary, description, steps to reproduce, current behavior, expected behavior and evidence. Then it either creates the ticket in Jira or returns it as plain text to paste

Postman and Newman: in my previous project, API tests were written in pytest and ran in a CI pipeline on every push. Here the deliverable was a Postman collection, so I adapted: the tests live inside the collection, and Newman runs the same file from the command line. That is also how it would go into CI, the same way pytest did

Evidence: every bug can be reproduced by running the Postman collection or the UI tests, which is why the report points to them instead of screenshots. With more time I would add screenshots and an end to end screen recording of each bug

Exploration notes: my raw notes from going through the UI, the API and the database, written as I tested, are in the repository at docs/exploratory-findings.md. This report was built from them

Test case management: in a real project I would keep the test cases in a TCMS like Kiwi TCMS or TestRail instead of a table in a document. That way the whole team can see them, edit them, comment on them and track the results of each run over time

Reports: I would add Allure on top of pytest. It gives a much clearer report, with a title, a description and the steps of each test, plus screenshots and traces when something fails. Here I kept short test names and the

default pytest report to keep the code simple for this scope

CI: a GitHub Actions workflow runs ruff, the UI tests and the Postman collection on every push to main. The API job is red on purpose while the known bugs are open, each failing assertion is a bug

9. AI NOTES (also in the repo /AI_NOTES.md)

How AI was used in this challenge and how its output was checked
Tool: Claude Code

What I used it for:
- Setting up the project faster: folder structure, Page Object base classes, pytest fixtures, env config, Postman auth script
- Reading the Swagger spec and the app to map endpoints and routes
- Exploring the database: schema, relations, appointment states and data anomalies (read-only queries)

What I decided myself:
- Stack (Python, pytest, Playwright, Page Object Model), one repo for everything, logging in through the API in UI tests, caching the token in Postman
- How to split the exploration: AI explored the database while I explored the web app by hand, so both happened in parallel and the app exploration stayed my own first-hand experience

How I validated it:
- Read and reviewed every file before keeping it
- Ran everything against the real environment: pytest (UI tests), Newman (Postman collection), ruff (lint)
- Tested pieces on their own, not only the full happy path
- Checked the key database findings myself in DBeaver (my appointments, the inactive doctor, the double-booked slot) before using them in the test strategy

Postman collection:
- I read the Swagger docs first to understand how the API works, then used that to guide Claude
- Claude generated the collection JSON and helped me decide the test cases and coverage, both the happy path and the edge cases
- I imported it into Postman, reviewed every request and test by hand, and refined it
- I ran it myself with Newman to confirm it works end to end and that every failure is a real bug

How I write tests with AI:
- First I give it context: the user story, my exploratory findings and the test cases I already wrote
- Then, for each test, I write the title and a short summary of what it should check and why. For example: "reschedule form requires date and time: a parametrized test with the input and the expected error message, so a failure shows which case broke"
- AI writes the code from that, following the project structure and conventions
- I review it, run it, and check that it fails for the right reason. Tests marked as expected failures were also run with --runxfail to confirm they fail because of the bug and not because of something else
- This can be improved having a command like /shape-test where we can add conventions to create test cases, so every test starts from the same template and I only write the intent